

Corso di Programmazione Web

Simulazione di esame – 16 Aprile 2025

- Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.
- Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.
- Fornire esempi di richieste e risposte per mostrare il funzionamento.
- Non scrivere codice effettivo, ma fornire dettagliate progettazioni API, i formati di richiesta/risposta e i modelli di dati.

Ci è stato chiesto di progettare un sistema per la gestione di eventi. Il sistema deve prevedere:

- Una API per creare un nuovo evento (con ID evento, titolo, descrizione, data, luogo);
- Una API per visualizzare l'elenco di eventi nel sistema;
- Una API per registrarsi a un evento come partecipante, indicando nome e cognome dell'utente.

Si osservi che l'evento deve essere identificato in modo univoco dal suo ID, mentre può avere gli altri campi non univoci. Si può assumere che ci sia solo un evento al giorno, per cui una stringa con la data può essere usata come ID, e viene creata automaticamente dal sistema (non passata dall'utente in fase di creazione). La risorsa evento può essere usata in diverse API. Si può assumere un sistema semplificato per cui agli utenti non è richiesta autenticazione.

Domanda 1: Definizione delle API (15 punti)

- Definire gli endpoint API (strutture URL e metodi HTTP); **(2.5 punti per ogni API)**
- Specificare i parametri di richiesta e le possibili risposte, possibilmente con un esempio di richiesta andata a buon fine; **(1.5 punti per ogni API)**
- Dettagliare i modelli di dati previsti per le richieste e le risposte. **(3 punti in totale)**

Domanda 2: Gestione degli Errori API (5 punti)

Una buona API deve gestire efficacemente gli errori.

- Descrivere le risposte di errore comuni (es. 400, 404, 500) e quando dovrebbero essere utilizzate nel contesto delle API appena progettate. **(3 punti)**
- Fornire esempi di risposta per almeno due casi di errore. **(1 punto per ogni esempio)**

Corso di Programmazione Web
Simulazione di esame – 16 Aprile 2025
Possibile soluzione

Si presenta di seguito una proposta di soluzione. Si osservi che questa presenta solo uno spunto di progettazione, ma non va intesa come l'unica soluzione possibile.

Soluzione alla Domanda 1: Design delle API

Endpoint:

- POST /events → Crea un evento.
- GET /events → Visualizza la lista di eventi.
- GET /events/{id} → Visualizza il singolo evento
- POST /events/{id}/register → Iscrizione a un evento.

Esempio:

POST /events

```
{
  "title": "Hackathon 2025",
  "description": "Evento di programmazione di 24 ore",
  "date": "2025-06-20",
  "location": "Milano"
}
```

Risposta:

```
{
  "id": "20250620",
  "message": "Evento creato con successo"
}
```

GET /events

```
[
  {
    "id": 101,
    "title": "Hackathon 2025",
    "date": "2025-06-20",
    "location": "Milano"
  }
]
```

GET /events/20250620

```
{
  "id": "20250620",
  "title": "Hackathon 2025",
  "date": "2025-06-20",
}
```

```
"location": "Milano"  
}
```

POST /events/20250620/register

```
{  
  "name": "Mario",  
  "surname": "Rossi",  
  "email": "mario.rossi@example.com"  
}
```

Soluzione alla Domanda 2: Gestione degli errori

Le risposte di errore sono usate dal server per comunicare all'utente diversi tipi di errori riscontrati durante l'esecuzione delle richieste.

- *Errore 400.* Questo errore caratterizza una richiesta errata da parte dell'utente. Per esempio, nel caso delle API appena progettate, potrebbe essere restituito qualora l'utente invii un indirizzo email non valido nel body della richiesta POST per la registrazione, o qualora invii una richiesta con il body vuoto.
- *Errore 404.* Questo errore è utilizzato per comunicare all'utente che non solo la richiesta è errata, ma che la risorsa richiesta non esiste. Può essere restituito da API a endpoint non esistenti (per esempio `/listaeventi/{id}`), o anche nel caso l'utente specifichi un parametro `{id}` non esistente nella lista di eventi.
- *Errore 500.* Questo errore viene usato per indicare un errore lato server. Per esempio, può essere restituito dall'API per l'inserimento di un nuovo evento se il server non è in grado di salvare l'informazione inviata dall'utente. Un altro esempio potrebbe essere qualora la lista di eventi diventasse troppo grande da gestire e facesse bloccare il server inavvertitamente.